

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امرالہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 2/30

Hello C++: Basic concepts, C++ compilers, and I/O Streams

- Hello C++: The first program: Hello, world!
- Install and Setup C++ compilers: GCC and Clang
- Compile, Link and Execute chain
- Basic concepts of I/O Stream
- Expressions and Statements
- Q & A

120 min (incl. Q & A)



Hello C++

Hello C++

Hello C++

```
// Hello world in Standard C++  
#include <iostream>  
  
int main()  
{  
    std::cout << "Hello, world!\n";  
  
    return 0;  
}
```

Hello C++

```
// Hello world in Standard C++
```

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    std::cout << "Hello, world!\n";
```

```
    return 0;
```

```
}
```

main function

Hello C++

```
// Hello world in Standard C++  
#include <iostream>  
  
int main()  
{  
    std::cout << "Hello, world!\n";  
  
    return 0;  
}
```

main function

C Global namespace

Hello C++

```
// Hello world in Standard C++
```

```
#include <iostream>
```

```
int main()
```

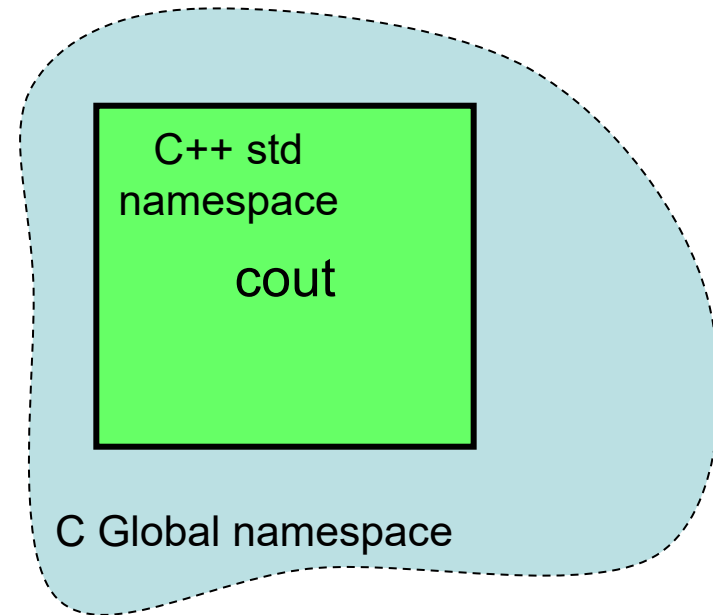
```
{
```

```
    std::cout << "Hello, world!\n";
```

```
    return 0;
```

```
}
```

main function



Hello C++

```
// Hello world in Standard C++
```

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    std::cout << "Hello, world!\n";
```

```
    return 0;
```

```
}
```

main function

C++ std
namespace

cout

C Global namespace

Hello C++

```
// Hello world in Standard C++
```

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    std::cout << "Hello, world!\n";
```

```
    return 0;
```

```
}
```

main function

C++ std
namespace

cout

C Global namespace

- *int* and *return* are C/C++ keywords.

Hello C++

```
// Hello world in Standard C++  
#include <iostream>  
  
int main()  
{  
    std::cout << "Hello, world!\n";  
    return 0;  
}
```

main function

C++ std
namespace

cout

C Global namespace

- *int* and *return* are C/C++ keywords.

Standard C++ = C++ programming language (core language) + standard library

Hello C++

```
// Hello world in Standard C++
#include <iostream>

int main()
{
    std::cout << "Hello, world!\n";
    return 0;
}
```

main function

C++ std
namespace

cout

C Global namespace

- *int* and *return* are C/C++ keywords.

Standard C++ = C++ programming language (core language) + standard library

Remember that standard library facilities are defined in namespace std.

The anatomy of Hello, world! program

■ Comments

- Single line comments

- //
- C++

```
// Hello world! in Standard C++
```

```
// This is a single line comment.
```

- Multi-line comments

- /*, */
- C/C++

```
/*  
    This is a multi-line comment.  
*/  
/* single line comment */
```

- multi-line comments and comments that end before the end of a line.

- The compiler ignores comments.

■ #include

```
#include <iostream> // get the standard I/O facilities
```

Angle brackets Standard header

- Programs ask for standard-library/non-standard library facilities using #include directive.

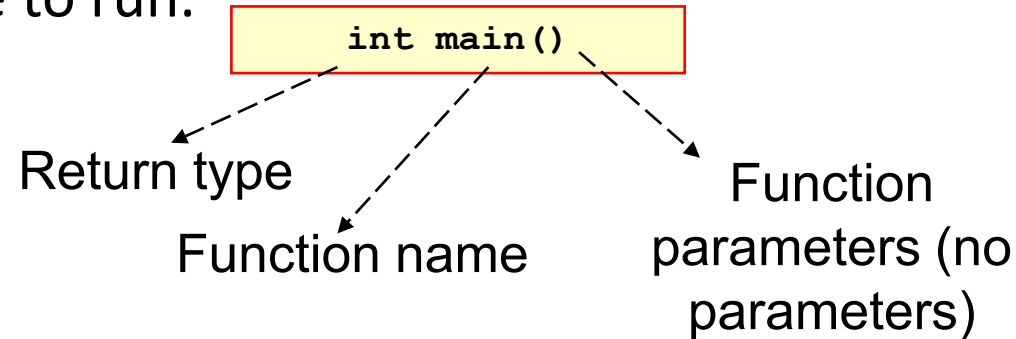
👉 Remember to #include the headers for the facilities you use;

The Main function

- Function: mathematical concept vs. Programming concept

$$y = f(x)$$

- Function: A **function** is a piece of program that has a name, and that another part of the program can *call*, or cause to run.



- Every C++ program must contain one and only a function called **main**.
- Curly braces
 - In C++, braces tell the implementation to treat whatever appears between them as a unit.

```
int main()
{ // the left brace
  // the statements go here
} // right brace
```

Standard library and output

- The name `iostream` suggests support for sequential, or stream, input-output, rather than random-access or graphical input-output

- Output operation:

- Output operator: `<<`

- put to

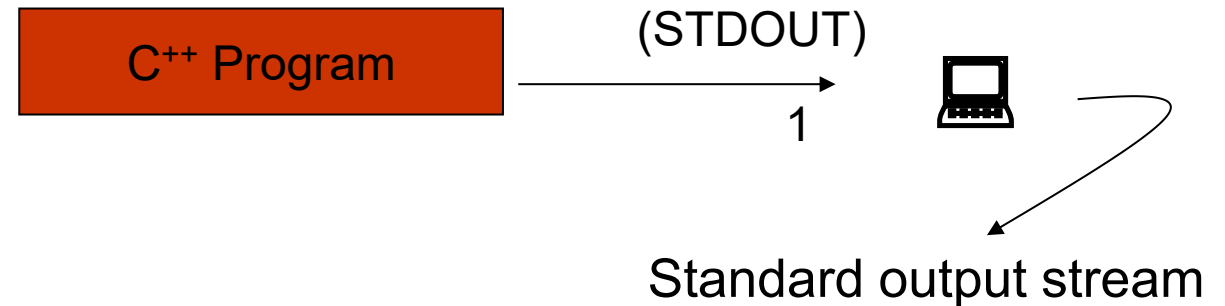
- Insertion operator (more technical sounding name)

- `cout`, an `ostream` class object

- namespace `std`

- `std::cout`

```
std::cout << "Hello, world!\n";
```



```
return 0; // 0 means success
```

- return statement
- return pass the value to operating system

C Compiling and Linking

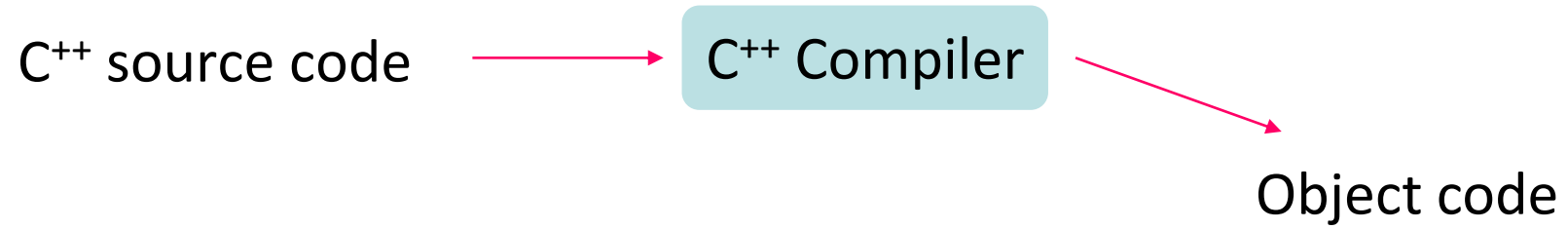
C Compiling and Linking

C++ source code

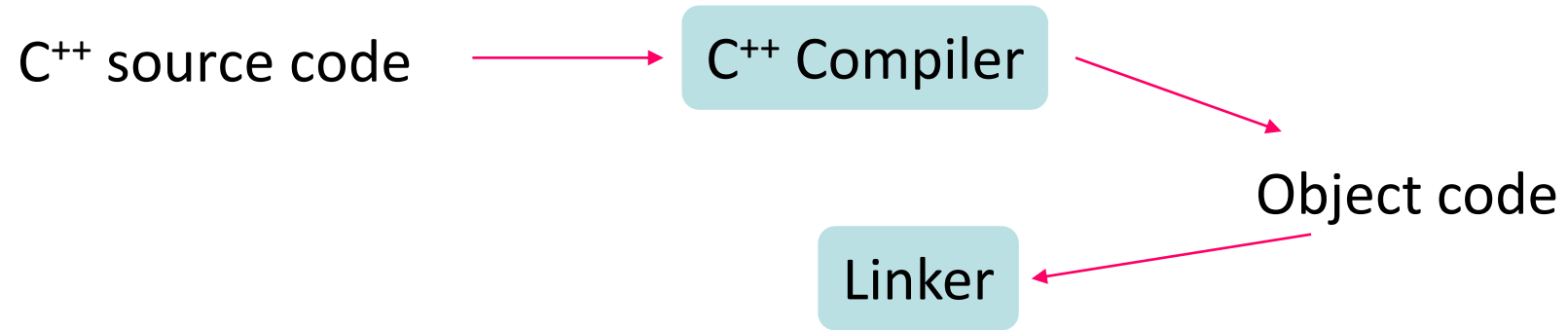


C++ Compiler

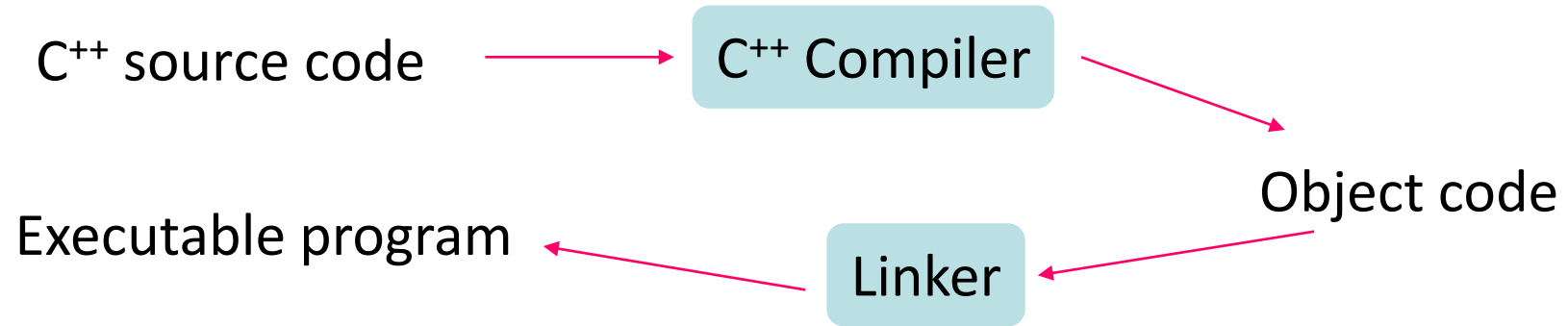
C Compiling and Linking



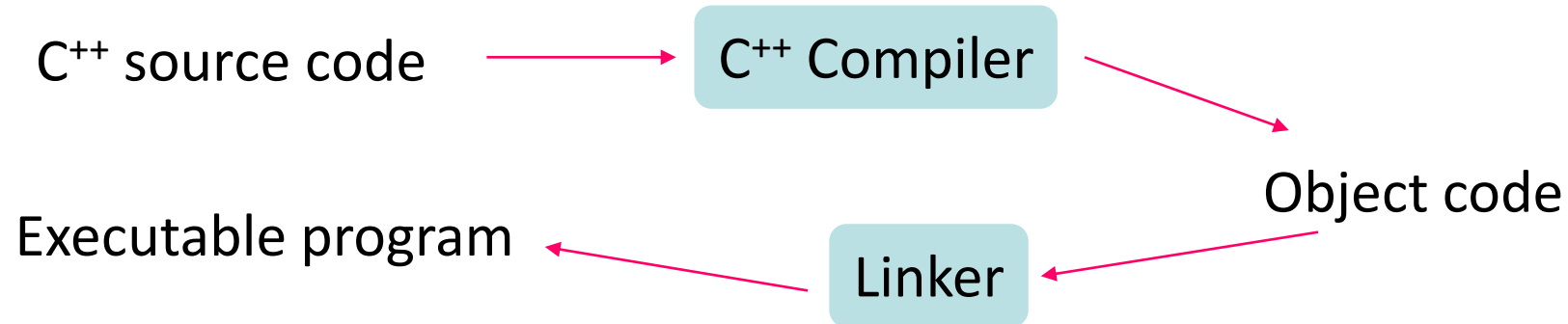
C Compiling and Linking



C Compiling and Linking

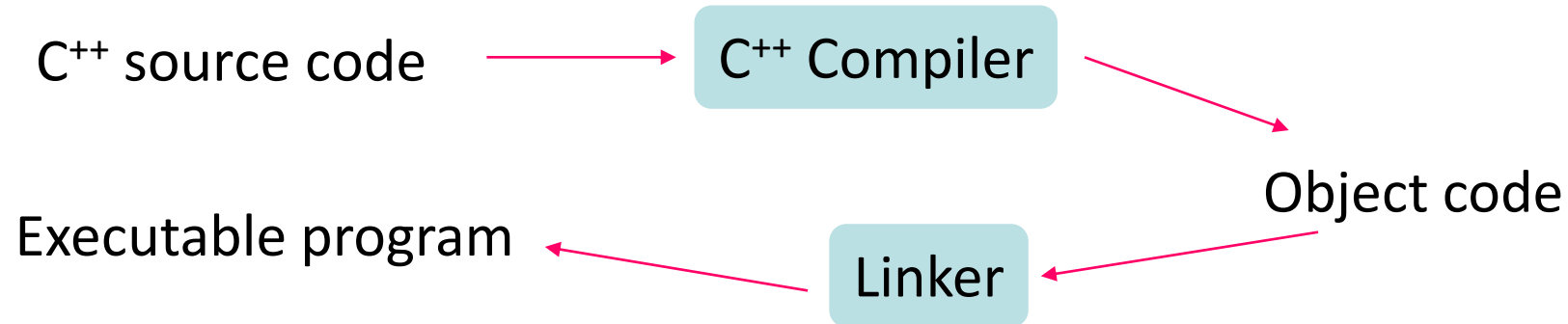


C Compiling and Linking



- You write **C++ source code** using a typical editor or IDE
 - Source code is (in principle) human readable
- The **compiler** translates what you wrote into **object code** (sometimes called machine code)
 - Object code is simple enough for a computer to “understand”
- The **linker** links your code to the system code needed to execute
 - E.g., input/output libraries, operating system code, and windowing code
- The result is an **executable program**
 - E.g., a .exe file on Windows or an a.out file on Unix

C Compiling and Linking



- You write **C++ source code** using a typical editor or IDE
 - Source code is (in principle) human readable
- The **compiler** translates what you wrote into **object code** (sometimes called machine code)
 - Object code is simple enough for a computer to “understand”
- The **linker** links your code to the system code needed to execute
 - E.g., input/output libraries, operating system code, and windowing code
- The result is an **executable program**
 - E.g., a .exe file on Windows or an a.out file on Unix

- GCC

```
$ g++ hello.cpp
$ ./a.out
Hello, world!
```

- Clang

```
$ clang++ hello.cpp
$ ./a.out
Hello, world!
```

Hello, world!: A deeper look

Hello, world!: A deeper look

- Expression

Hello, world!: A deeper look

- Expression
 - An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

Hello, world!: A deeper look

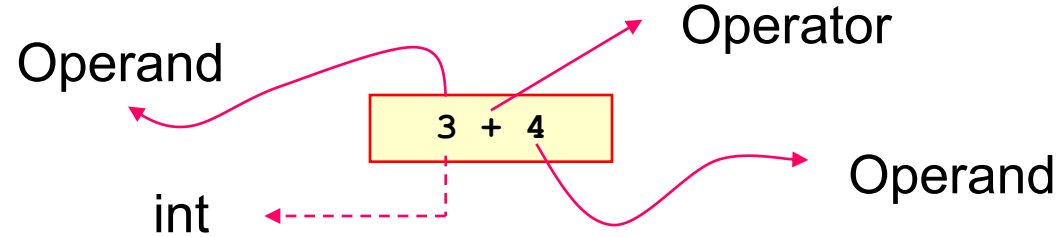
- Expression
 - An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

```
3 + 4 // result 7, no side effect
```

Hello, world!: A deeper look

- Expression
 - An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

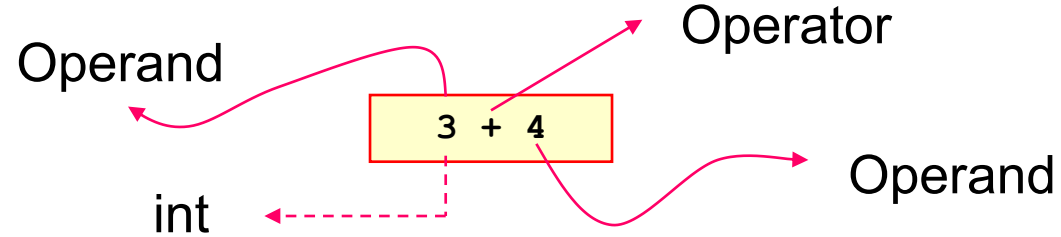
```
3 + 4 // result 7, no side effect
```



Hello, world!: A deeper look

- Expression
 - An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

```
3 + 4 // result 7, no side effect
```

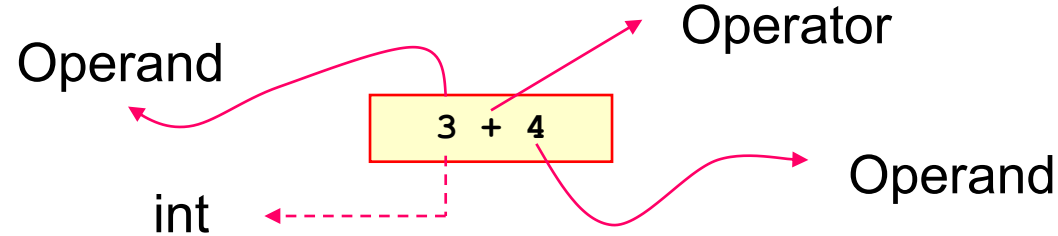


```
std::cout << "Hello, world!" << '\n'; // no result, has side effect
```

Hello, world!: A deeper look

- Expression
 - An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

```
3 + 4 // result 7, no side effect
```



```
std::cout << "Hello, world!" << '\n'; // no result, has side effect
```

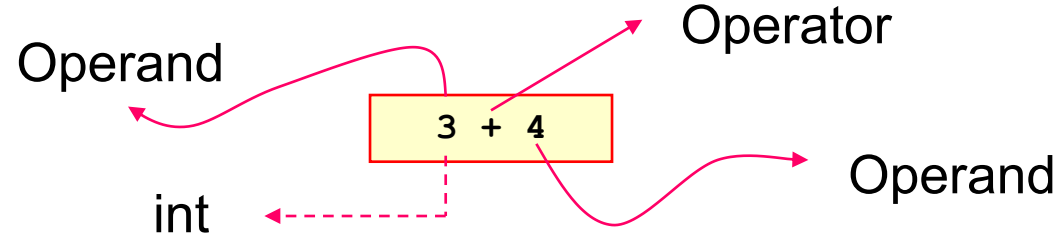
- As its side effect, writes Hello, world! on the standard output streams and ends the current line.

Hello, world!: A deeper look

- Expression

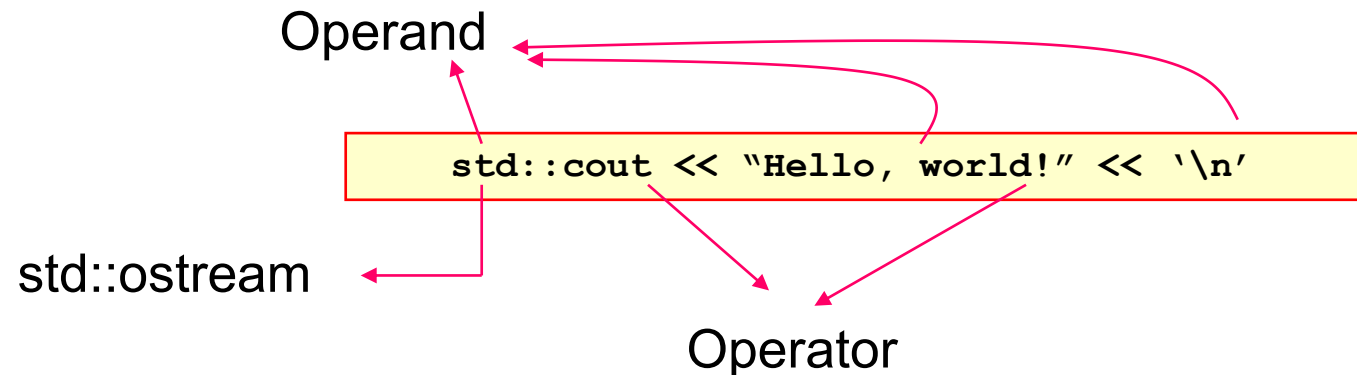
- An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

```
3 + 4 // result 7, no side effect
```



```
std::cout << "Hello, world!" << '\n'; // no result, has side effect
```

- As its side effect, writes Hello, world! on the standard output streams and ends the current line.

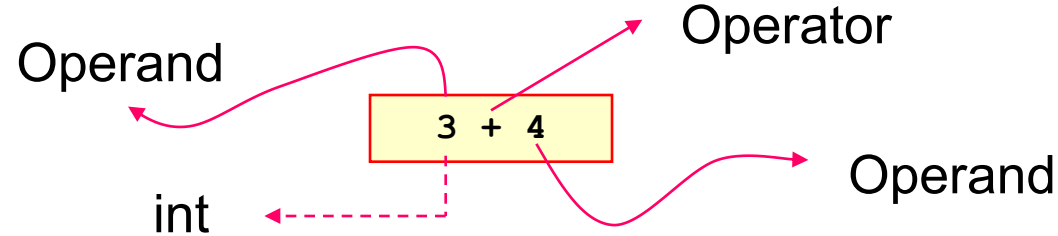


Hello, world!: A deeper look

- Expression

- An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

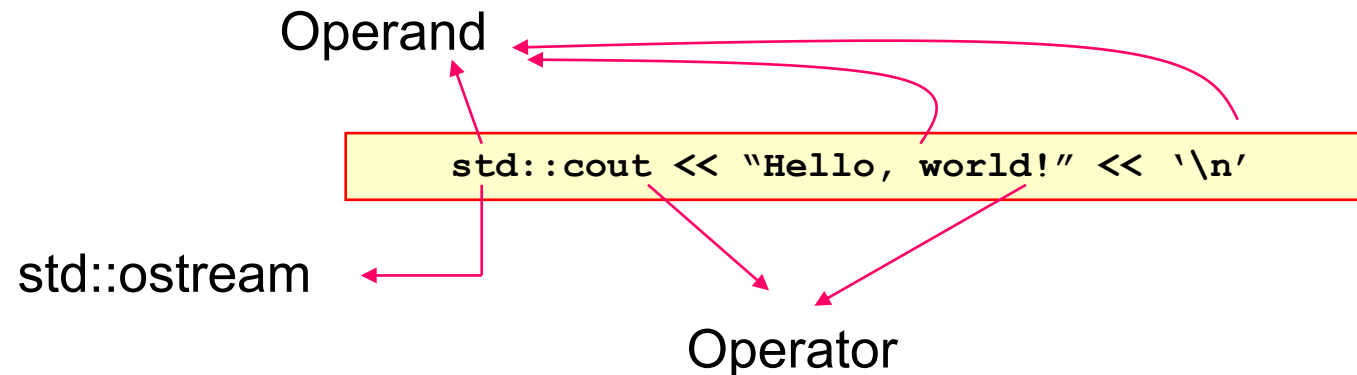
```
3 + 4 // result 7, no side effect
```



```
std::cout << "Hello, world!" << '\n'; // no result, has side effect
```

- As its side effect, writes Hello, world! on the standard output streams and ends the current line.

- Precedence and Association

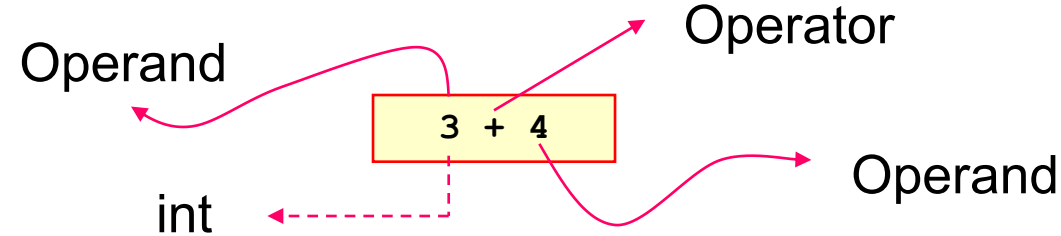


Hello, world!: A deeper look

- Expression

- An **expression** asks the implementation to compute something. The computation yields a **result**, and may also have **side effects**.

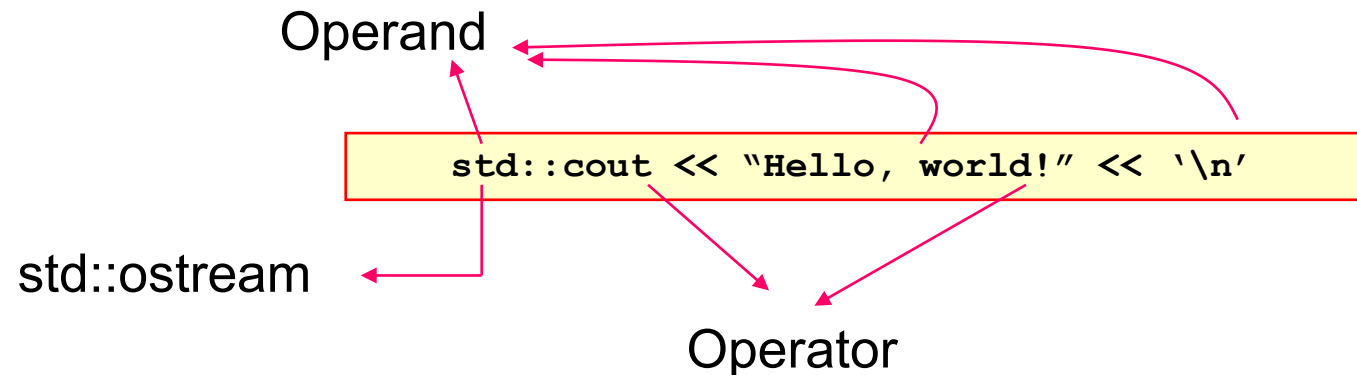
```
3 + 4 // result 7, no side effect
```



```
std::cout << "Hello, world!" << '\n'; // no result, has side effect
```

- As its side effect, writes Hello, world! on the standard output streams and ends the current line.

- Precedence and Association
- Every operand has a **type**.



Hello, world!: A deeper look, cont.

HHello, world!: A deeper look, cont.

- Precedence and Association

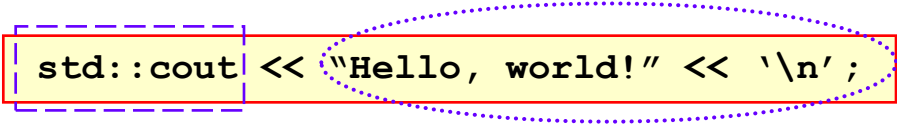
HHello, world!: A deeper look, cont.

- Precedence and Association
 - << is left-associative

Hello, world!: A deeper look, cont.

- Precedence and Association
 - << is left-associative

```
std::cout << "Hello, world!" << '\n';
```



Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

```
3 + 4 // expression
```

```
3 + 4; // statement
```

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

```
3 + 4 // expression
```

```
3 + 4; // statement
```

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

- \n is a **manipulator**. A manipulator, manipulates a stream.

```
3 + 4 // expression
```

```
3 + 4; // statement
```

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

- \n is a **manipulator**. A manipulator, manipulates a stream.

```
3 + 4 // expression
```

```
3 + 4; // statement
```

- Scope

- The **scope** of a name is the part of a program in which that name has its meaning.

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

- \n is a **manipulator**. A manipulator, manipulates a stream.

```
3 + 4 // expression
```

```
3 + 4; // statement
```

- Scope

- The **scope** of a name is the part of a program in which that name has its meaning.

- Block

- Compound statement = Block

Hello, world!: A deeper look, cont.

- Precedence and Association

- << is left-associative

```
std::cout << "Hello, world!" << '\n';
```

```
std::cout << "Hello, world!"; // return: nothing, side effect: output  
std::cout << '\n'; // return: nothing, side effect: go to next line
```

- Statement vs. expression

- \n is a **manipulator**. A manipulator, manipulates a stream.

```
3 + 4 // expression
```

```
3 + 4; // statement
```

- Scope

- The **scope** of a name is the part of a program in which that name has its meaning.

- Block

- Compound statement = Block

Left curly brace

← {

```
/* ... */
```

Right curly brace

← }

R references and further readings

- [KM00] Andrew Koenig and Barbara Moo. Accelerated C++: Practical Programming in C++, Addison Wesley, 2000.
- [Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.
- [Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

